Module 2 – Introduction to Programming

1. Write an essay covering the history and evolution of C programming. Explain its importance and why it is still used today.

C is a powerful programming language developed in 1972 by Dennis Ritchie at Bell Labs. It was mainly created to develop the UNIX operating system. Before C, programmers used assembly language, which was difficult and depended on the machine. C made programming easier because it is simple, fast, and portable.

C was developed from an earlier language called B. It combined the power of low-level programming with the simplicity of high-level languages. Because of this, programmers could control hardware directly while still writing readable code.

Importance of C

C is important because:

- It is fast and efficient.
- It allows direct access to memory and hardware.
- Many operating systems, compilers, and embedded systems are written in C.
- It is the base language for many other languages like C++, Java, and Python.

Why C is Still Used Today

C is still widely used today because:

- It is reliable and performs well.
- It is used in system programming, embedded systems, and device drivers.

- It works close to the hardware, which is important for performance.
- Many old systems and software are written in C and still need maintenance.
- It helps programmers understand how computers work internally.
- Describe the steps to install a C compiler (e.g., GCC) and set up an Integrated Development Environment (IDE) like DevC++, VS Code, or CodeBlocks.

❖LAB EXERCISE:

❖Research and provide three real-world applications where C programming is extensively used, such as in embedded systems, operating systems, or game development.

2. Describe the steps to install a C compiler (e.g., GCC) and set up an Integrated Development Environment (IDE) like DevC++, VS Code, or CodeBlocks.

1. Install GCC Compiler

Download MinGW from the official website.

- Install it and select gcc packages.

- Add the path C:\MinGW\bin to Environment Variables.
- Open Command Prompt and type:

gcc --version

- If version shows, GCC is installed.

2. Install an IDE

- Option 1: Dev-C++
- Download and install.
- Open → New File → Write code → Press F11 to run.
- (Compiler is already included)
- Option 2: Code::Blocks.
- Download Code::Blocks with MinGW.
- Install → Create Console Project → Build & Run (F9).

Option 3: Visual Studio Code

Install VS Code.

Install C/C++ extension.

- LAB EXERCISE:

- Install a C compiler on your system and configure the IDE. Write your first program to print "Hello, World!" and run it.

```c
#include <stdio.h>

int main() {
    printf("Hello, World!");
    return 0;
}
```

Output :

```
Hello, World!
--------------------------------
Process exited after 0.0787 seconds with return value 0
Press any key to continue . . .
```

3. THEORY EXERCISE:

- Explain the basic structure of a C program, including headers, main function, comments, data types, and variables. Provide examples.

- A C program has the following parts:

1. **Header Files**

   Used to include library functions.
   ;Example:

```
#include <stdio.h>
```

2. **Main Function**
   The starting point of every C program.

```
int main() {
    // code
    return 0;
}
```

3. **Comment**

   Used to explain the code. They are not executed.

```
// This is a single-line comment
/* This is a multi-line comment */
```

4. **Data Types**

   Define the type of data.
   Examples: `int`, `float`, `char`

5. **Variables**

Used to store values.
Example:

```
int a = 10;
float b = 5.5;
char c = 'A';
```

Example Program

```
#include <stdio.h>   // Header file

int main() {
    int a = 10;       // Variable
declaration
    printf("Value = %d", a);   // Output
    return 0;
}
Output :

       a=10
```

- LAB EXERCISE:

- Write a C program that includes variables, constants, and comments. Declare and use different data types (int, char, float) and display their values.

```c
#include <stdio.h>
#define PI 3.14   // Constant using #define
main() {
    int age = 20;           // Integer variable
    float height = 5.9;     // Float variable
    char grade = 'A';       // Character variable
    const int year = 2026; // Constant using const keyword
    // Displaying values
    printf("Age: %d\n", age);
    printf("Height: %.2f\n", height);
    printf("Grade: %c\n", grade);
    printf("Year: %d\n", year);
    printf("Value of PI: %.2f\n", PI);
}
```

Output :

```
Age: 20
Height: 5.90
Grade: A
Year: 2026
Value of PI: 3.14

--------------------------------
Process exited after 0.09073 seconds with return value 18
Press any key to continue . . .
```

4. THEORY EXERCISE:

Write notes explaining each type of operator in C: arithmetic, relational, logical, assignment, increment/decrement, bitwise, and conditional operators.

1. **Arithmetic Operators :**

Used to perform mathematical operations.
Operators: `+ - * / %`
Example:

```
int a = 10, b = 5;
int c = a + b;   // c = 15
```

---

2. Relational Operators :

Used to compare two values. Result is true (1) or false (0).
Operators: `== != > < >= <=`
Example:

```
if (a > b)
```

---

3. Used to combine conditions :

Operators: `&& (AND), || (OR), ! (NOT)`
Example:

```
if (a > 5 && b < 10)
```

---

4. Assignment Operators :

Used to assign values to variables.
Operators: `= += -= *= /= %=`
Example:

```
int a = 5;
a += 2;   // a = 7
```

---

5. Increment/Decrement Operators :

Used to increase or decrease a value by 1.
Operators: `++ --`
Example:

```
a++;    // increase by 1
b--;    // decrease by 1
```

---

6. **Bitwise Operators :**

   Used to perform operations on bits.
   Operators: `& | ^ ~ << >>`
   Example:

```
int c = a & b;
```

---

7. **Conditional (Ternary) Operator** :

   Works like a short form of if-else.
   Operator: `?:`
   Example:

```
int max = (a > b) ? a : b;
```

❖ LAB EXERCISE:

❖Write a C program that accepts two integers from the user and performs arithmetic, relational, and logical operations on them. Display the results.

```c
#include <stdio.h>
main() {
    int a, b;
    // Taking input from user
    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);
    // Arithmetic Operations
    printf("\n--- Arithmetic Operations ---\n");
    printf("Addition: %d\n", a + b);
    printf("Subtraction: %d\n", a - b);
    printf("Multiplication: %d\n", a * b);
    if (b != 0) {
        printf("Division: %d\n", a / b);
        printf("Modulus: %d\n", a % b);
    } else {
        printf("Division and Modulus not possible (division by zero)\n");
    }
    // Relational Operations
    printf("\n--- Relational Operations ---\n");
    printf("a == b : %d\n", a == b);
    printf("a != b : %d\n", a != b);
    printf("a > b  : %d\n", a > b);
    printf("a < b  : %d\n", a < b);
    printf("a >= b : %d\n", a >= b);
    printf("a <= b : %d\n", a <= b);
    // Logical Operations
    printf("\n--- Logical Operations ---\n");
    printf("(a > 0 && b > 0) : %d\n", (a > 0 && b > 0));
    printf("(a > 0 || b > 0) : %d\n", (a > 0 || b > 0));
    printf("!(a > 0) : %d\n", !(a > 0));
}
```

Output :

```
Enter two integers: 12
24

--- Arithmetic Operations ---
Addition: 36
Subtraction: -12
Multiplication: 288
Division: 0
Modulus: 12

--- Relational Operations ---
a == b : 0
a != b : 1
a > b  : 0
a < b  : 1
a >= b : 0
a <= b : 1

--- Logical Operations ---
(a > 0 && b > 0) : 1
(a > 0 || b > 0) : 1
!(a > 0) : 0

--------------------------------
Process exited after 7.314 seconds with return value 13
Press any key to continue . . .
```

5. THEORY EXERCISE:

- Explain decision-making statements in C (if, else, nested if-else, switch). Provide examples of each.

1. if Statement

Used to execute a block if the condition is true.

```
#include <stdio.h>
int main() {
```

```
    int a = 10;
    if (a > 5) {
        printf("a is greater than 5");
    }
    return 0;
}
Output :
          a is greater than 5
```

---

2. if-else Statement

Used to choose between two options.

```
#include <stdio.h>
int main() {
    int a = 3;
    if (a % 2 == 0) {
        printf("Even");
    } else {
        printf("Odd");
    }
    return 0;
}
Output :
           odd
```

---

3. Nested if-else

An if-else inside another if-else.

```
#include <stdio.h>
int main() {
    int a = 10;
    if (a > 0) {
        if (a % 2 == 0)
            printf("Positive Even");
        else
            printf("Positive Odd");
```

```
    } else {
        printf("Negative or Zero");
    }
    return 0;
}
Output :
         Positive Even
```

---

4. switch Statement

Used to select one option from many cases.

```
#include <stdio.h>
int main() {
    int day = 2;
    switch(day) {
        case 1: printf("Monday"); break;
        case 2: printf("Tuesday"); break;
        case 3: printf("Wednesday"); break;
        default: printf("Invalid day");
    }
    return 0;
}
Output :
         Tuesday
```

❖ LAB EXERCISE:

❖ Write a C program to check if a number is even or odd using an if-else statement. Extend the program using a switch statement to display the month name based on the user's input (1 for January, 2 for February, etc.).

```c
#include <stdio.h>
main() {
    int num, month;
    // Even or Odd using if-else
    printf("Enter a number: ");
    scanf("%d", &num);

    if (num % 2 == 0) {
        printf("The number is Even.\n");
    } else {
        printf("The number is Odd.\n");
    }
   // Month name using switch
    printf("\nEnter month number (1-12): ");
    scanf("%d", &month);

    switch (month) {
        case 1:
            printf("January\n");
            break;
        case 2:
            printf("February\n");
            break;
        case 3:
            printf("March\n");
            break;
        case 4:
            printf("April\n");
            break;
        case 5:
            printf("May\n");
            break;
        case 6:
            printf("June\n");
            break;
        case 7:
            printf("July\n");
            break;
        case 8:
            printf("August\n");
            break;
        case 9:
            printf("September\n");
            break;
        case 10:
            printf("October\n");
            break;
        case 11:
            printf("November\n");
            break;
        case 12:
            printf("December\n");
            break;
        default:
            printf("Invalid month number!\n");
    }
}
```

Output :

```
Enter a number: 34
The number is Even.

Enter month number (1-12): 4
April

--------------------------------
Process exited after 8.153 seconds with return value 0
Press any key to continue . . .
```

6. THEORY EXERCISE:

Compare and contrast while loops, for loops, and do-while loops. Explain the scenarios in which each loop is most appropriate.

➢ Loops are used to repeat a block of code.

---

1. while Loop

- Checks the condition **before** execution.
- Executes only if the condition is true.
- Use when the number of iterations is **not known**.

Example:

```
int i = 1;
while(i <= 5) {
    printf("%d ", i);
    i++;
}
```

---

2. for Loop

- Condition, initialization, and increment in one line.
- Use when the number of iterations is **known**.

Example:

```
for(int i = 1; i <= 5; i++) {
    printf("%d ", i);
}
```

---

3. do-while Loop

- Executes the loop **at least once**.
- Condition is checked **after** execution.
- Use when the code must run at least one time.

Example:

```
int i = 1;
do {
    printf("%d ", i);
    i++;
} while(i <= 5);
```

➢ Difference

Loop	Condition Check	Best Use
while	Before loop	Unknown iterations
for	Before loop	Known iterations
do-while	After loop	Must run at least once

❖ LAB EXERCISE:

❖ Write a C program to print numbers from 1 to 10 using all three types of loops (while, for, do-while).

7. THEORY EXERCISE:

Explain the use of break, continue, and goto statements in C. Provide examples of each.

➢ These statements control the flow of loops and programs.

---

1. break Statement

- Used to **exit a loop or switch** immediately.

Example:

```
#include <stdio.h>
int main() {
    for(int i = 1; i <= 5; i++) {
        if(i == 3)
            break;
        printf("%d ", i);
    }
    return 0;
}
```

Output: 1 2

---

2. continue Statement

- Skips the current iteration and moves to the **next loop cycle**.

Example:

```
#include <stdio.h>
int main() {
    for(int i = 1; i <= 5; i++) {
        if(i == 3)
            continue;
        printf("%d ", i);
    }
    return 0;
}
```

Output: 1 2 4 5

---

3. goto Statement

- Transfers control to a labeled statement.
- Generally **not recommended**, but useful in some cases.

Example:

```
#include <stdio.h>
int main() {
    int i = 1;
start:
    printf("%d ", i);
    i++;
```

```
    if(i <= 5)
        goto start;
    return 0;
}
```

- ❖ LAB EXERCISE:
- ❖ Write a C program that uses the break statement to stop printing numbers when it reaches 5. Modify the program to skip printing the number 3 using the continue statement.

8. THEORY EXERCISE:

   What are functions in C? Explain function declaration, definition, and how to call a function. Provide examples.

A **function** is a block of code that performs a specific task. It helps make the program simple, reusable, and easy to understand.

---

1. Function Declaration (Prototype)

- ➢ It tells the compiler the function name, return type, and parameters.

```
int add(int, int);
```

---

2. Function Definition

- ➢ It contains the actual code of the function.

```
int add(int a, int b) {
    return a + b;
}
```

---

3. Function Call

- ➢ It is used to execute the function.

```
int sum = add(5, 3);
```

Example Program

```
#include <stdio.h>

// Function declaration
int add(int, int);

int main() {
    int result;
    result = add(5, 3);   // Function call
    printf("Sum = %d", result);
    return 0;
}

// Function definition
int add(int a, int b) {
    return a + b;
}
```

- ❖ LAB EXERCISE:
- ❖ Write a C program that calculates the factorial of a number using a function. Include function declaration, definition, and call.

9. THEORY EXERCISE:

Explain the concept of arrays in C. Differentiate between one-dimensional and multi-dimensional arrays with examples.

- ➢ An **array** is a collection of elements of the same data type stored in continuous memory locations. It is used to store multiple values using a single variable name.

Syntax:

```
datatype array_name[size];
```

Example:

```
int a[5];
```

---

1. One-Dimensional Array (1D)

- Stores data in a single row (list form).
- Accessed using one index.

Example:

```
#include <stdio.h>
int main() {
    int a[5] = {10, 20, 30, 40, 50};
    printf("%d", a[2]);   // Output: 30
    return 0;
}
```

---

2. Multi-Dimensional Array (2D)

- Stores data in rows and columns (table form).
- Accessed using two indexes.

Example:

```
#include <stdio.h>
int main() {
    int a[2][2] = {{1, 2}, {3, 4}};
    printf("%d", a[1][0]);   // Output: 3
    return 0;
}
```

Difference Between 1D and Multi-Dimensional Arrays

1D Array	Multi-Dimensional Array
Single list of elements	Table (rows and columns)
One index	Two or more indexes

❖ LAB EXERCISE:

❖ Write a C program that stores 5 integers in a one-dimensional array and prints them. Extend this to handle a two-dimensional array (3x3 matrix) and calculate the sum of all elements.

10. THEORY EXERCISE:

Explain what pointers are in C and how they are declared and initialized. Why are pointers important in C?

➢ A **pointer** is a variable that stores the **memory address** of another variable.

---

1. Declaration of Pointer

Use `*` to declare a pointer.

```
int *p;
```

This means `p` is a pointer to an integer.

---

2. Initialization of Pointer

Assign the address of a variable using `&`.

```
int a = 10;
int *p = &a;
```

- `&a` gives the address of `a`
- `p` stores that address

To access the value:

```
printf("%d", *p);   // prints 10
```

`*p` is called **dereferencing**.

Example Program

```
#include <stdio.h>
int main() {
    int a = 5;
    int *p = &a;
    printf("Value = %d", *p);
    return 0;
}
```

Why Pointers are Important

- Used for **dynamic memory allocation**
- Help in **passing arguments by reference**
- Needed for **arrays, strings, and structures**
- Used in **system programming and memory management**
- Makes programs **faster and efficient**

- ❖ LAB EXERCISE:
- ❖ Write a C program to demonstrate pointer usage. Use a pointer to modify the value of a variable and print the result.

11. THEORY EXERCISE:

Explain string handling functions like strlen(), strcpy(), strcat(), strcmp(), and strchr(). Provide examples of when these functions are useful.

- ➢ These functions are available in the header file:

```
#include <string.h>
```

---

1. strlen()

- Finds the **length of a string**.

Example:

```
char str[] = "Hello";
printf("%d", strlen(str));   // Output: 5
```

Use: To count characters in a string.

---

2. strcpy()

- Copies one string into another.

Example:

```
char a[10];
char b[] = "Hi";
strcpy(a, b);
```

Use: To copy text from one variable to another.

---

3. strcat()

- Joins (concatenates) two strings.

Example:

```
char a[20] = "Hello ";
char b[] = "World";
strcat(a, b);   // a = "Hello World"
```

Use: To combine two strings.

---

4. strcmp()

- Compares two strings.
- Returns 0 if equal.

Example:

```
if(strcmp("abc", "abc") == 0)
    printf("Equal");
```

Use: To check if two strings are the same.

---

5. strchr()

- Finds the **first occurrence of a character** in a string.

Example:

```
char str[] = "Hello";
char *p = strchr(str, 'l');
printf("%s", p);   // Output: llo
```

Use: To search for a character in a string.

❖ LAB EXERCISE:

❖ Write a C program that takes two strings from the user and concatenates them using strcat(). Display the concatenated string and its length using strlen().

12. THEORY EXERCISE:

Explain the concept of structures in C. Describe how to declare, initialize, and access structure members.

- A **structure** is a user-defined data type used to store different types of data together (like int, float, char) under one name.

---

1. Declaration of Structure

```
struct Student {
    int id;
    char name[20];
    float marks;
};
```

---

2. Creating Structure Variable

```
struct Student s1;
```

---

3. Initialization

```
struct Student s1 = {1, "John", 85.5};
```

---

4. Accessing Structure Members

Use the **dot (.) operator**.

```
printf("%d", s1.id);
printf("%s", s1.name);
printf("%f", s1.marks);
```

Example Program

```
#include <stdio.h>

struct Student {
    int id;
    char name[20];
    float marks;
};

int main() {
    struct Student s1 = {1, "John", 85.5};

    printf("ID: %d\n", s1.id);
    printf("Name: %s\n", s1.name);
    printf("Marks: %.2f", s1.marks);
```

```
    return 0;
}
```

- LAB EXERCISE:
- Write a C program that defines a structure to store a student's details (name, roll number, and marks). Use an array of structures to store details of 3 students and print them.

13 THEORY EXERCISE:

Explain the importance of file handling in C. Discuss how to perform file operations like opening, closing, reading, and writing files.

➢ Importance of File Handling

➢ File handling is used to **store data permanently** in a file.
It helps to:

- Save data for future use
- Read stored data later
- Avoid data loss when the program ends
- Handle large amounts of data

Steps for File Operations

1. Opening a File

Use `fopen()` function.

```
FILE *fp;
fp = fopen("data.txt", "w");
```

Modes:

- `"r"` → read
- `"w"` → write
- `"a"` → append

---

2. Writing to a File

Use `fprintf()` or `fputs()`.

```
fprintf(fp, "Hello World");
```

---

3. Reading from a File

Use `fscanf()` or `fgets()`.

```
char str[20];
fscanf(fp, "%s", str);
```

---

4. Closing a File

Always close the file using `fclose()`.

```
fclose(fp);
```

Example Program

```
#include <stdio.h>
int main() {
    FILE *fp;
    fp = fopen("data.txt", "w");
    fprintf(fp, "Hello C");
    fclose(fp);
    return 0;
}
```

- ❖ LAB EXERCISE:
- ❖ Write a C program to create a file, write a string into it, close the file, then open the file again to read and display its contents.